

# SKILL - 16

---

**Name:** Simran Jangid

**Rollno:** 2520030166

**Section:** 8

---

## Aim

To detect background jobs, launch non-blocking processes, return the shell prompt immediately, store job information, monitor execution, and test background tasks. Also, to create a job table, store process groups, track process states, update job status, remove completed jobs, and validate job records.

## Objective

- To detect background job requests.
- To launch non-blocking child processes.
- To return the prompt immediately after starting a background job.
- To store information about running jobs.
- To monitor background process execution.
- To test background tasks.
- To create and manage a job table.
- To store process group information.
- To track job states and update their status.
- To remove completed jobs from the table.
- To validate stored job records.

## Problem Statement

Design and implement a shell-style background job manager that can identify commands requested for background execution, launch them without blocking the main shell, return the prompt immediately, store process and job information, monitor process completion, and update or remove job records as execution changes.

## Theory

A shell normally waits for a foreground process to complete before accepting another command. Background execution allows a command to run independently while the shell immediately returns to its prompt. The shell must therefore maintain information about background processes so that their progress can be monitored and their records can be updated when they finish.

### 1. Background Jobs

A background job is a command launched without blocking the main command loop. In a shell, a command ending with an ampersand can indicate that the user wants the process to continue in the background.

## 2. Non-Blocking Process Launch

After creating a child process using `fork()`, the child can execute the requested program while the parent does not wait immediately for completion. This allows the shell prompt to return without delay.

## 3. Immediate Prompt Return

The parent shell can display the prompt as soon as the background process is launched. The shell therefore remains responsive while the child process continues execution.

## 4. Job Information

A job record can store a job identifier, process ID, process-group ID, command name, and current state. This information allows the shell to identify and monitor each active background task.

## 5. Process Groups

A process group groups related processes under a common process-group identifier. Storing this identifier is useful when a job may involve more than one process and when the shell needs to manage the job as a unit.

## 6. Job States

A background job may move through states such as `RUNNING`, `COMPLETED`, or `FAILED`. The job table is updated when the operating system reports that a child process has changed state.

## 7. Job Monitoring

The shell periodically checks background processes using non-blocking wait operations. When a child process has completed, the corresponding job record is updated and the completed job can be removed from the active table.

## 8. Job Table

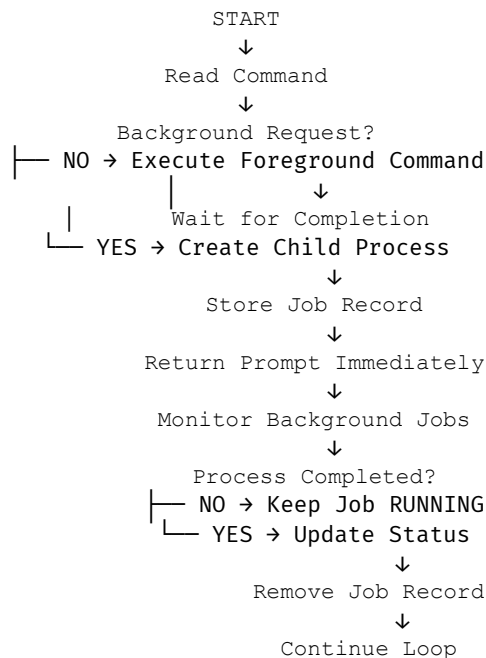
A job table is an array or other data structure containing the records of currently tracked background jobs. Each record can contain the job ID, PID, process-group ID, state, and command text.

## 9. Completed Job Removal

When a background process finishes, its job entry should no longer remain in the active table. Removing completed entries keeps the job table consistent and prevents stale records.

## 10. Control Flow

The complete control flow is kept on this page so that the flowchart is not divided across pages.



## 11. Algorithm

1. Start the program.
2. Initialize an empty job table.
3. Read a command from the user.
4. Check whether the command requests background execution.
5. If it is a foreground command, execute it and wait for completion.
6. If it is a background command, create a child process using `fork()`.
7. In the child, create a separate process group if required and execute the command.
8. In the parent, store the job ID, PID, process-group ID, command, and `RUNNING` state.
9. Return the prompt immediately without waiting for the background job.
10. Periodically check each stored process using a non-blocking wait operation.
11. If a process is still active, keep its job state as `RUNNING`.
12. If a process has completed, update its state to `COMPLETED` or `FAILED`.
13. Remove the completed job record from the active job table.
14. Display job completion information.
15. Continue accepting and monitoring commands.
16. Stop when the shell receives the exit command.

## 12. Data Used

A job structure stores the job ID, process ID, process-group ID, state, and command text. An array of such structures forms the job table. Integer values store counts and indexes, while process IDs identify operating-system processes.

## 13. Operations Used

- `fork()` for creating child processes.
- `setpgid()` for creating or assigning process groups.
- `execvp()` for executing the background command.
- `waitpid()` with `WNOHANG` for non-blocking monitoring.
- `kill()` or process-status checks for monitoring where required.
- Job-table insertion and removal operations.
- State updates for `RUNNING`, `COMPLETED`, and `FAILED` jobs.

## 14. Job Table Structure

| Job ID | PID  | PGID | State     | Command |
|--------|------|------|-----------|---------|
| [1]    | 4211 | 4211 | RUNNING   | sleep 5 |
| [2]    | 4212 | 4212 | RUNNING   | sleep 8 |
| [3]    | 4213 | 4213 | COMPLETED | date    |

## 15. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 20

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char state[20];
    int active;
} Job;

Job jobs[MAX_JOBS];
int next_job_id = 1;

void add_job(pid_t pid, const char *command) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active) {
            jobs[i].active = 1;
            jobs[i].job_id = next_job_id++;
            jobs[i].pid = pid;
            jobs[i].pgid = pid;
            strcpy(jobs[i].command, command);
            strcpy(jobs[i].state, "RUNNING");

            printf("[%d] %d running in background\n",
                jobs[i].job_id, jobs[i].pid);
            return;
        }
    }

    printf("Job table is full.\n");
}

void monitor_jobs() {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active)
            continue;

        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == 0) {
            continue;
        }

        if (result == jobs[i].pid) {
            if (WIFEXITED(status) &&
                WEXITSTATUS(status) == 0) {
                strcpy(jobs[i].state, "COMPLETED");
            } else {
                strcpy(jobs[i].state, "FAILED");
            }

            printf("[%d] %s %s\n",
                jobs[i].job_id,
                jobs[i].state,
                jobs[i].command);

            jobs[i].active = 0;
        }
    }
}

void show_jobs() {
    printf("\nActive Jobs:\n");

    int found = 0;
```

```

for (int i = 0; i < MAX_JOBS; i++) {
    if (jobs[i].active) {
        printf("[%d] PID=%d PGID=%d STATE=%s CMD=%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            jobs[i].state,
            jobs[i].command);
        found = 1;
    }
}

if (!found)
    printf("No active background jobs.\n");
}

int main() {
    char input[100];

    printf("Skill - 16: Background Job Management\n");
    printf("-----\n");

    while (1) {
        monitor_jobs();

        printf("\nskill16> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            monitor_jobs();
            printf("Exiting Skill-16 shell...\n");
            break;
        }

        if (strcmp(input, "jobs") == 0) {
            show_jobs();
            continue;
        }

        int background = 0;
        int len = strlen(input);

        if (len > 0 && input[len - 1] == '&') {
            background = 1;
            input[len - 1] = '\0';

            while (len > 0 && input[len - 2] == ' ') {
                input[len - 2] = '\0';
                len--;
            }
        }

        char *args[20];
        int count = 0;

        char *token = strtok(input, " ");

        while (token != NULL && count < 19) {
            args[count++] = token;
            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        if (count == 0)
            continue;

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            continue;
        }
    }
}

```

```

    }

    if (pid == 0) {
        setpgid(0, 0);
        execvp(args[0], args);
        perror(args[0]);
        exit(1);
    }

    if (background) {
        setpgid(pid, pid);
        add_job(pid, input);
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("Foreground command completed.\n");
    }
}

return 0;
}

```

## 16. Program Explanation

### 16.1 Job Structure

The Job structure stores the job ID, PID, process-group ID, command text, state, and active flag. This provides the information needed to validate and monitor each background job.

### 16.2 Background Detection

The program checks whether the command ends with '&'. When present, the command is launched as a background task and the parent does not wait immediately.

### 16.3 Job Table

`add_job()` inserts a new record into the job table. Each active job receives a unique job ID and is initially stored with the RUNNING state.

### 16.4 Monitoring

`monitor_jobs()` uses `waitpid()` with `WNOHANG`. This allows the shell to check whether a child has finished without blocking the command prompt.

### 16.5 Job Removal

When a process completes, its state is updated to COMPLETED or FAILED and its active flag is cleared. This removes the completed job from the active job table.

### 16.6 Process Groups

The child creates its own process group using `setpgid()`. The process-group ID is stored in the job record for management and validation.

## 17. Sample Output

Skill - 16: Background Job Management

-----

```
skill16> sleep 3 &
[1] 4211 running in background
```

```
skill16> jobs
```

Active Jobs:

```
[1] PID=4211 PGID=4211 STATE=RUNNING CMD=sleep 3
```

```
skill16> date
Sat Aug 23 16:00:00 IST 2026
Foreground command completed.

skill16>
[1] COMPLETED sleep 3

skill16> jobs
No active background jobs.

skill16> exit
Exiting Skill-16 shell...
```

## 18. Output Screenshot

Paste the screenshot of the program output in the space provided below.

```
Skill - 16: Background Job Management
-----

skill16> sleep 5 &
jobs
date
jobs
sleep 2 &
jobs
exit[1] PID=2095 PGID=2095 started in background

skill16>
Active Jobs:
[1] PID=2095 PGID=2095 STATE=RUNNING CMD=sleep

skill16> Sun Aug 23 16:16:01 IST 2026
Foreground command completed.

skill16>
Active Jobs:
[1] PID=2095 PGID=2095 STATE=RUNNING CMD=sleep

skill16> [2] PID=2097 PGID=2097 started in background

skill16>
Active Jobs:
[1] PID=2095 PGID=2095 STATE=RUNNING CMD=sleep
[2] PID=2097 PGID=2097 STATE=RUNNING CMD=sleep

skill16>
[2] COMPLETED: sleep
Exiting Skill-16 shell...
7rainreddy7@Gunapatis-MacBook-Air ~ %
```

## 19. Testing

- Tested a background command using the '&' operator.
- Verified that the prompt returned without waiting for completion.
- Verified that a job record was stored with PID and PGID.
- Used the jobs command to inspect active job records.
- Verified RUNNING state while the process was active.
- Verified COMPLETED state after the process ended.
- Verified that completed jobs were removed from the active table.
- Tested a foreground command while a background job was running.

## 20. Test Cases

| Test Case | Input / Scenario     | Expected Result                                       |
|-----------|----------------------|-------------------------------------------------------|
| 1         | sleep 3 &            | Background job starts and prompt returns immediately. |
| 2         | jobs                 | Active job record is displayed.                       |
| 3         | sleep 2 & then jobs  | Job state is shown as RUNNING before completion.      |
| 4         | After sleep finishes | Job becomes COMPLETED and is removed.                 |
| 5         | date                 | Foreground command executes normally.                 |
| 6         | exit                 | Shell exits after checking job state.                 |

## 21. Observation

- Background commands were identified using the '&' operator.
- Child processes were launched without blocking the parent shell.
- The prompt returned immediately after background-job creation.
- Each background task received a job ID, PID, and process-group ID.
- The job state remained RUNNING while the process was active.
- The state changed to COMPLETED or FAILED after process termination.
- Completed jobs were removed from the active job table.
- Job records could be inspected and validated using the jobs command.

## 22. Result

Background job detection and management were successfully implemented. The program launched non-blocking child processes, returned the prompt immediately, stored job information including PID and process-group ID, monitored process states, updated job records, and removed completed jobs from the active table.

## 23. Conclusion

The practical demonstrated how an operating system shell manages background processes without blocking user interaction. Job tables, process groups, non-blocking wait operations, state updates, and cleanup work together to provide reliable background-task management.